# Day 3 — Artificial Intelligence and Data Science

## Topics Covered

1. Introduction to Pandas
2. Pandas Series
3. Pandas DataFrames — Part 1
4. Creating Series and DataFrames
5. Accessing columns
6. Inspecting datasets
7. Basic numerical analysis
8. Preparing tabular data for Machine Learning

Tools Used:

- Python
- Pandas
- NumPy (previous learning)
- Google Colab

---

## 1. What Is Pandas?

Pandas is an open-source Python library used for data manipulation and analysis. It provides convenient structures for working with labelled, tabular, and time-series data.

Pandas is commonly used for:

- Loading datasets.
- Organizing data into tables.
- Selecting rows and columns.
- Handling missing values.
- Filtering data.
- Grouping and summarizing data.
- Preparing datasets for Machine Learning.

### Importing Pandas

import pandas as pd

`pd` is the conventional short alias for the Pandas library.

---

## 2. Why Do We Need Pandas?

Python lists can store data, but analyzing structured datasets often requires additional code.

For example, suppose we have student names and marks.

```
names = ["Deep", "Aman", "Priya", "Rahul"]
marks = [90, 85, 92, 78]
```

Pandas helps us organize related information into a table and perform calculations more conveniently.

```
import pandas as pd

data = {
    "Name": ["Deep", "Aman", "Priya", "Rahul"],
    "Marks": [90, 85, 92, 78]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
   Name  Marks
0   Deep     90
1   Aman     85
2  Priya     92
3  Rahul     78
```

The table contains labelled columns and an index for each row.

---

## 3. What Is a Pandas Series?

A Series is a one-dimensional labelled array that can store values such as integers, floating-point numbers, strings, and other Python objects.

## Creating a Series

```
import pandas as pd

marks = pd.Series([80, 90, 75, 88, 95])

print(marks)
```

Output:

```
0    80
1    90
2    75
3    88
4    95
dtype: int64
```

Explanation:

- The left column contains index labels.
- The right column contains the values.
- Pandas creates a default index starting at 0.
- `dtype` indicates the data type used for the stored values.

## Accessing Series Elements

```
print(marks[0])
print(marks[2])
```

Output:

```
80
75
```

## Basic Series Operations

```
print("Mean:", marks.mean())
print("Maximum:", marks.max())
print("Minimum:", marks.min())
print("Sum:", marks.sum())
print("Count:", marks.count())
```

Expected results:

```
Mean: 85.6
Maximum: 95
Minimum: 75
```

Sum: 428
Count: 5

### Creating a Series with Custom Index Labels

```
import pandas as pd

marks = pd.Series(
    [90, 85, 92],
    index=["Deep", "Aman", "Priya"]
)

print(marks)
print(marks["Deep"])
```

Output:

```
Deep     90
Aman     85
Priya    92
dtype: int64

90
```

Custom labels can make data easier to understand and access.

---

## 4. What Is a DataFrame?

A DataFrame is a two-dimensional labelled data structure consisting of rows and columns. Different columns can contain different data types.

It is similar to a table in a spreadsheet or a database.

### Creating a DataFrame

```
import pandas as pd

data = {
    "Name": ["Deep", "Aman", "Priya", "Rahul"],
    "Age": [19, 20, 19, 21],
    "Marks": [90, 85, 92, 78],
    "Branch": ["AI", "CSE", "AI", "CSE"]
}
```

```
df = pd.DataFrame(data)

print(df)
```

Output:

```
   Name  Age  Marks Branch
0   Deep   19     90     AI
1   Aman   20     85    CSE
2  Priya   19     92     AI
3  Rahul   21     78    CSE
```

### Explanation

- Each dictionary key becomes a column name.
- Each list provides values for that column.
- Each row represents one record.
- All columns must have compatible lengths when constructing a DataFrame from lists.

---

# 5. Accessing DataFrame Columns

### Access One Column

```
print(df["Name"])
print(df["Marks"])
```

### Access Multiple Columns

```
print(df[["Name", "Marks"]])
```

The double square brackets are used to provide a list of column names.

### Access a Row by Position

```
print(df.iloc[0])
```

`iloc` selects rows or columns by integer position.

### Access a Value Using Row and Column Labels

```
print(df.loc[0, "Name"])
```

`loc` selects data using index and column labels.

Note: In a DataFrame with a custom index, `loc` uses labels, whereas `iloc` uses integer positions.

---

# 6. Important DataFrame Attributes and Methods

## A. head()

Displays the first five rows by default.

print(df.head())

To display the first two rows:

print(df.head(2))

## B. tail()

Displays the last five rows by default.

print(df.tail())

## C. shape

Returns the number of rows and columns as a tuple.

print(df.shape)

For the four-row, four-column dataset:

(4, 4)

## D. columns

Displays the column labels.

print(df.columns)

## E. dtypes

Shows the data type of each column.

print(df.dtypes)

### F. describe()

Generates descriptive statistics for suitable numeric columns.

print(df.describe())

It can display count, mean, standard deviation, minimum, quartiles, and maximum.

### G. info()

Displays information about columns, non-null counts, data types, and memory usage.

df.info()

Important distinction:

- `shape`, `columns`, and `dtypes` are attributes.
- `head()`, `tail()`, `describe()`, and `info()` are methods.

---

# 7. Performing Calculations on a DataFrame

Pandas makes it easy to calculate statistics on a numerical column.

```
print("Average marks:", df["Marks"].mean())
print("Highest marks:", df["Marks"].max())
print("Lowest marks:", df["Marks"].min())
print("Total marks:", df["Marks"].sum())
print("Number of records:", df["Marks"].count())
```

Expected results for the sample dataset:

```
Average marks: 86.25
Highest marks: 92
Lowest marks: 78
Total marks: 345
Number of records: 4
```

These calculations are useful for understanding a dataset before using it for analysis or Machine Learning.

---

# 8. Filtering Data

Filtering selects records that satisfy a condition.

### Students Scoring More Than 80

```
high_scores = df[df["Marks"] > 80]

print(high_scores)
```

### Students from the AI Branch

```
ai_students = df[df["Branch"] == "AI"]

print(ai_students)
```

### Combining Conditions

```
result = df[(df["Marks"] > 80) & (df["Branch"] == "AI")]

print(result)
```

Important:

- Use `&` for element-wise AND between conditions.
- Use `|` for element-wise OR.
- Put each condition inside parentheses.

---

# 9. NumPy and Pandas — The Difference

NumPy and Pandas are both important for numerical computing and data science, but they serve different purposes.

| Feature | NumPy | Pandas |
|---|---|---|
| Main structures | ndarray | Series and DataFrame |
| Main use | Numerical arrays and mathematical operations | Tabular data manipulation and analysis |
| Labels | Usually position-based | Supports labelled indexes and columns |
| Typical data | Numerical arrays, matrices | Tables with named rows and columns |

| Example | `np.array([10, 20, 30])` | `pd.DataFrame(data)` |
|---|---|---|

NumPy is especially useful for numerical calculations. Pandas builds on array-based computing concepts and adds convenient tools for labelled and tabular data.

---

# 10. How Pandas Helps in Machine Learning

Pandas is often used before model training to prepare and understand a dataset.

A typical workflow is:

1. Load the dataset.
2. Inspect rows, columns, and data types.
3. Calculate summary statistics.
4. Identify missing values.
5. Filter or transform records.
6. Separate features and target variables.
7. Prepare data for a Machine Learning library.

### Example: Preparing Features and Target

```
import pandas as pd

data = {
   "StudyHours": [2, 4, 5, 7, 8],
   "Attendance": [60, 70, 75, 90, 95],
   "Marks": [50, 60, 68, 85, 92]
}

df = pd.DataFrame(data)

X = df[["StudyHours", "Attendance"]]
y = df["Marks"]

print("Features:")
print(X)

print("Target:")
print(y)
```

Explanation:

- `X` contains the input features.
- `y` contains the target we want to predict.
- Here, study hours and attendance are used as example input features, and marks are the example target.

This is a basic illustration of data preparation, not a trained prediction model.

---

# 11. Practical Exercises

## Exercise 1: Series Practice

Create a Series containing the marks of five students.

Tasks:

1. Print the Series.
2. Calculate the mean.
3. Find the maximum mark.
4. Find the minimum mark.
5. Calculate the sum.

## Exercise 2: DataFrame Practice

Create a DataFrame with at least five students.

Columns:

- Name
- Age
- Marks
- Branch

Tasks:

1. Display the full DataFrame.
2. Display the first three rows.
3. Print the shape.
4. Print the column names.
5. Display only the Name column.
6. Calculate the average marks.
7. Find the highest marks.
8. Filter students scoring above 80.
9. Filter students belonging to the AI branch.

10. Display descriptive statistics.

### Exercise 3: Modify the Dataset

Add two more student records and repeat the calculations.

Observe how the shape, average, maximum, and filtered results change.

---

## 12. Important Revision Questions

1. What is Pandas?
2. Why is Pandas useful for Data Science?
3. What is a Series?
4. What is a DataFrame?
5. What is the difference between a Series and a DataFrame?
6. What does `pd.Series()` do?
7. What does `pd.DataFrame()` do?
8. What is the purpose of `head()`?
9. What does `shape` return?
10. What is the difference between `loc` and `iloc`?
11. What does `describe()` show?
12. How can we select a single DataFrame column?
13. How can we filter rows based on a condition?
14. How do NumPy and Pandas differ?
15. How can Pandas help prepare a dataset for Machine Learning?

---

## 13. Day 3 Summary

### Concepts Learned

- Introduction to Pandas.
- Creating and accessing Series.
- Creating DataFrames from dictionaries.
- Selecting columns and rows.
- Inspecting data using `head()`, `shape`, `columns`, `dtypes`, `describe()`, and `info()`.
- Calculating basic statistics.
- Filtering data using conditions.
- Preparing input features and target variables for Machine Learning.

## Practical Work

- Created a Pandas Series.
- Calculated mean, maximum, minimum, and sum.
- Created a student DataFrame.
- Inspected and filtered the dataset.
- Practised basic data preparation in Google Colab.

## Next Topics

- DataFrames Part 2 and Part 3.
- Missing Data.
- Groupby.
- Merging, Joining, and Concatenating.
- DataFrame Operations.
- Data Input and Output.

## Main Takeaway

Pandas provides practical tools for organizing, inspecting, filtering, and analyzing structured data. These skills form an important foundation for Data Science and Machine Learning.